ADOBE UNIVERSITY HACKATHON 2026 · GRAND FINALE PREP

Brand AI-Readiness Audit, explained from basics to advanced

A study guide to your Agent Skill Marketplace: the problem statements, every concept, every skill, the severity and priority maths, the testing, the weak spots a judge could find, and how to answer them.

Covers: Round 3 and Round 4 problem statements · `brand-ai-readiness-audit/` (9 skills) · Query Intelligence extension · real runs on adobe.com and iiitm.ac.in
Verified on 2026-10-08: all 525 detector unit tests pass. No engine code was modified.

Contents

1 The problem in plain English

2 Basic concepts you must be fluent in

3 The big idea: discovery is a funnel

4 Architecture: what happens in one run

5 Each skill in detail

6 The brain: severity, correlation and priority

7 Precision engineering and testing

8 Safety and honesty

9 Query Intelligence (optional extension)

10 Real results you can cite

11 Weak spots a judge could find, and honest answers

12 Finale prep: pitch, Q&A and demo hazards

> **How to use this.** Parts 1–4 give you the mental model. Part 5 is the reference for each skill. Part 6 is the part judges probe hardest, so know its two worked examples by heart. Parts 11 and 12 are your finale prep. File references are relative to `brand-ai-readiness-audit/` unless shown otherwise.

1 · The problem in plain English

When someone asks ChatGPT "best laptop for students under ₹60,000", the assistant often searches, fetches a few pages, builds an answer and cites some of them. A brand can be missing from that answer even when its website has the exact information, because a **machine** failed somewhere along the way. A human visiting the site would never notice.

Round	What was asked
Round 2	Explain on paper why a brand is invisible, stale or losing visitors in AI apps.
Round 3	Turn that reasoning into an **Agent Skill Marketplace**. Point it at *any* site and it audits it and emits a report: findings (evidence + severity) plus prioritized fixes. It is graded on sites you have never seen.
Round 4	Show it working live on camera with an AI agent in a CLI, and prove it is reproducible with a `REPLAY_<Team>.txt` file. The engine must be identical to Round 3.

The problem has two halves:

1. **Off-site discoverability:** why AI assistants don't find or cite the brand.
2. **On-site engagement:** why visitors who do arrive don't stay or continue.

What the Round 3 report had to contain

- Per finding: `id`, `title`, `severity`, `evidence`, `suggested_action`.
- Report metadata: `site`, `audited_at`, and counts by severity.
- Rubric: detection accuracy, suggested-action quality, output design, skill-format hygiene, marketplace composition, generalization.
- Guardrails: recommend-only, read-only, respect robots.txt, no authenticated or destructive actions, under 5 minutes per audit, zip under 50 MB.

How Round 4 was judged

Criterion	Weight	Looking for
Reasoning (Part 1 of video)	35	Signal discovery, severity logic and fix derivation are sound, non-obvious and traceable to the code.
Live behaviour (Part 2)	35	On an unseen URL, real evidence-backed findings, few misses, few false positives.
Reproducibility & hygiene	30	Judges can replay via REPLAY.txt and reach the same substance; deterministic; fully disclosed harness and model.

Gates applied regardless of score: integrity (no fabricated findings, same engine as Round 3, no spliced runs), unverified (judges can't reproduce), and a nondeterminism flag.

2 · Basic concepts you must be fluent in

Term	What it is	Why it matters in your project
Crawler / user-agent	A program that fetches pages. It names itself in the `User-Agent` header (e.g. `GPTBot`).	Different bots do different jobs (next row).
Training vs retrieval vs user-triggered bots	*Training* bots (GPTBot, ClaudeBot, Google-Extended) collect data to train future models. *Retrieval* bots (OAI-SearchBot, PerplexityBot, Googlebot) build the indexes today's answers come from. *User-triggered* bots (ChatGPT-User) fetch a page because a user asked about it.	**Your headline insight.** Blocking training bots costs nothing in citations. Blocking retrieval bots removes you from AI answers entirely.
robots.txt	A file listing `User-agent` groups with `Allow` / `Disallow` rules.	The most specific matching group applies, and within it the longest matching rule wins. Naive substring matching gets this wrong.
sitemap.xml	A list of the site's URLs with `lastmod` dates.	Helps discovery. If every entry has the same `lastmod`, it is a build timestamp, not a real change date.
noindex / X-Robots-Tag	Tells search indexes "don't index this page".	Fine on cart and search pages, a defect on content pages.
canonical	`<link rel=canonical>` names the official URL for duplicate content.	If missing or conflicting, signals get split across duplicate URLs.
Raw HTML vs rendered DOM	Raw HTML is what the server sends. The rendered DOM is what a browser builds after running JavaScript.	Most AI crawlers **don't run JavaScript**. A React/SPA "shell" page looks empty to them.
Structured data	Machine-readable labels in JSON-LD, Microdata or RDFa using the schema.org vocabulary (`Product`, `Organization`, `@id`, `sameAs`).	Helps machines know *what* a page is and *whose* it is. It improves eligibility; it does **not** guarantee citation.
Entity / mistaken identity	The real-world "thing" a page describes.	If two brands share a name, `sameAs` links and a stable `@id` help disambiguate.
Quotability	Whether a fact is stated plainly, near the top, with units, with the entity named.	An assistant lifts sentences. A fact that is buried or has no unit loses to a cleaner page.
Agent Skill	A folder with a `SKILL.md` (YAML frontmatter: `name`, `description`, `license`, `allowed-tools`, then instructions), plus optional `scripts/` and `references/`.	The agentskills.io format. Keep `SKILL.md` lean and push detail into `references/` ("progressive disclosure").
Marketplace	`marketplace.json` lists every skill and marks **exactly one** entrypoint.	The contest's own convention. Yours lists 9 skills with 1 entrypoint.

3 · The big idea: discovery is a funnel

```
exists → discovered → accessible → readable → understood → retrieved → selected
       → (visitor arrives) → engagement
```

A page can pass one gate and fail the next, and a fix aimed at the wrong gate does nothing. Perfect structured data on a page that robots.txt blocks is worth zero. So the design is **one detector per gate**:

Gate	Question a machine asks	Skill (namespace)
discovered, accessible	Am I allowed in, and can I find the pages?	`crawl-access-audit` (ACC)
readable	Can I read the text without running JavaScript?	`render-readability-audit` (RND)
understood	What is this, and which brand does it belong to?	`structured-data-entity-audit` (SDE)
retrieved, selected	Can I lift out and quote the exact fact?	`answer-retrievability-audit` (ANS)
selected	Which version of the fact wins, and is it current?	`freshness-corroboration-audit` (FRC)
engagement	Once a human lands, is there a next step?	`engagement-audit` (ENG)

Plus three non-detector skills: `brand-ai-readiness-audit` (the entrypoint, which owns no checks), `site-snapshot-collector` (the only network-touching skill) and `opportunity-and-monitoring` (synthesis).

> **Why this is not padding:** each detector reads a different slice of the evidence, produces a different kind of finding, has a different fix, and could be owned by a different engineer. The rubric rewards "genuine separation of concerns" and penalizes padding.

4 · Architecture: what happens in one run

```
URL
 │
 ▼
site-snapshot-collector ──► snapshot.json      ← the ONLY step that touches the network
 │
 ▼  six detectors, each a pure function over snapshot.json (no network)
ACC → RND → SDE → ANS → FRC → ENG  ──► findings-<skill>.json ×6
 │
 ▼
opportunity-and-monitoring ──► opportunities.json  (cross-finding synthesis, capped at 8)
 │
 ▼
merge_findings.py   → severity, dedup, escalation, priority, action bands → report.json
validate_report.py  → schema + "evidence-or-silence" checks
render_report_md.py → report.md (plain English for a non-expert)
```

Why capture one snapshot first

1. **Politeness and budget.** The site is crawled once, not six times.
2. **Consistency.** All detectors see exactly the same page, so they can't disagree about what the site said.
3. **Evidence integrity.** Every finding describes the exact state that was audited.
4. **Determinism and safety.** Detectors can't make network calls, which shrinks the attack surface.

The contract is frozen in `skills/site-snapshot-collector/references/snapshot-contract.md`. Its rule: *if a detector needs something the snapshot lacks, that dimension is `not_assessed`, never a finding.*

Detectors don't assign severity

They emit *severity inputs* (`gate_impact`, `reach`, `certainty`), and the entrypoint decides. That keeps the severity logic in one place: `skills/brand-ai-readiness-audit/scripts/merge_findings.py`.

How the entrypoint composes the other skills

The Agent Skills spec doesn't define skill-to-skill calls, so `references/composition-protocol.md` gives three fallback paths:

1. activate the sibling skill natively;
2. read `../<skill>/SKILL.md` and follow it;
3. run the detector scripts directly with `--snapshot` / `--out`.

`audit.py` is path 3 automated. It is a convenience runner with no audit logic of its own.

The entrypoint's six phases (SKILL.md)

Phase	What happens
0 · Precheck	Normalise the URL, refuse non-http(s), localhost and private IPs, state the read-only contract.
1 · Acquire	Run the collector once to produce `snapshot.json`.

Phase	What happens
2 · Detect	Run the six detectors in funnel order.
3 · Correlate	Dedup, escalate corroborating pairs, demote behind a shut gate, cap by certainty.
4 · Uplift	opportunity-and-monitoring: at most 8 opportunities plus the measurement plan.
5 · Rank	Priority by funnel precedence, reach, effort; three action bands.
6 · Emit	Build `report.json`, validate, render `report.md`, give a short spoken summary.

5 · Each skill in detail

5.1 site-snapshot-collector (acquisition)

- **robots.txt** (`scripts/robots.py`):
  - picks the group by the longest matching agent token, falling back to `*`;
  - within a group, the longest rule wins and `Allow` wins a tie;
  - supports `*` and `$` patterns;
  - **records the deciding rule and its line number** so a finding can quote it.
- **Discovery:** reads sitemaps from robots.txt, then `/sitemap.xml`. If there is no sitemap, it crawls in-domain links breadth-first to depth 3. It never guesses URLs.
- **Clustering into templates:** `/products/velocity-x9` becomes `/products/{slug}`. A second pass merges sibling leaves (`/products/alpha.html` and `/products/beta.html` are one template).
- **Sampling** (deterministic):
  1. the homepage always;
  2. up to 3 pages per template, breadth-first, with nav-linked clusters ranked up (so About and Contact get sampled) and faceted URLs ranked down;
  3. leftover budget goes round-robin to the clusters.

  Members are sorted, so a rerun picks the same pages. That is why it generalizes: it tests every *template*, not the first 25 near-identical URLs.
- **Double fetch:**
  - an honest audit user-agent (`BrandAIReadinessAudit/1.0`);
  - a **control fetch** with a browser user-agent, used only to detect "the site blocks bots but serves browsers". It is disclosed in the output and still obeys the `*` robots group (`collect.py:759`).
- **Optional render:**
  - if Playwright is present, it renders one page per top template (max 6);
  - **it never clicks or dismisses anything**; a JavaScript probe records modal overlays structurally (no keywords);
  - without a browser it records `not_assessed` and computes static SPA signals instead.
- **Budgets** (`scripts/config.json`): 120 requests, 150 s, 3 MB per response; concurrency 4, dropping to 1 after a 429/503; one retry; `Crawl-delay` honoured.
- **Rule: degrade, never abort.** Every fallback is in `references/degradation-modes.md`.

5.2 crawl-access-audit (21 checks)

> **Headline: role, not vendor, decides whether a block is a defect.** The role table is `skills/crawl-access-audit/references/agent-roles.json`.

- `ACC.ROBOTS_BLOCKS_RETRIEVAL`: a retrieval or user-triggered bot is disallowed. A real block.
- `ACC.TRAINING_ONLY_BLOCK`: training bots blocked, retrieval allowed. Pinned to **info**; the action says "no change required".

- `ACC.ROBOTS_BLOCKS_ALL`: `*` disallows `/`. Suppressed if named retrieval bots are explicitly allowed, because that is an allowlist strategy, not a blackout.
- `ACC.UA_BLOCKED`: 401/403 to the audit agent but 2xx to the browser control fetch. It requires the control fetch to succeed; otherwise it is "our access problem", recorded as coverage.
- Also: noindex (utility pages excluded), sitemap missing/dead/uniform lastmod, status errors, deep links that redirect to the homepage, locale redirect traps, canonical missing/cross-domain/conflict, hreflang (only on sites that publish it).
- Unknown bot tokens map to `unknown` and are never the basis of a finding. **The failure mode is silence, not a guess.**

Example: the negative-control fixture `F03-robots-blocks-training-only` blocks GPTBot, ClaudeBot, Google-Extended and CCBot while allowing OAI-SearchBot, Claude-SearchBot and PerplexityBot. Its expectation file allows only `ACC.TRAINING_ONLY_BLOCK` at `info`, and forbids `ROBOTS_BLOCKS_RETRIEVAL` and `ROBOTS_BLOCKS_ALL`.

5.3 render-readability-audit (17 catalogue entries, 15 can emit)

- `RND.CSR_SHELL`:
  - **with a browser:** raw main text < 150 characters, and rendered text ≥3× larger and ≥80 characters;
  - **without one:** two high-weight static signals fire it as "blocks"; one high plus one medium fires it as "degrades";
  - the fallback path sets confidence to `medium`, which caps it at high (`site-snapshot-collector/scripts/render.py:153`).
- `RND.PARTIAL_JS_CONTENT`: the spec table appears only after JavaScript. `RND.JS_ONLY_STRUCTURED_DATA`: JSON-LD injected by JavaScript.
- `RND.FACTS_IN_IMAGES` and `RND.FACTS_IN_DOCUMENTS_ONLY`: specs only in images without alt text, or only in PDFs.
- `RND.INTERSTITIAL_IN_RAW`: a dialog marker plus thin text, meaning a crawler receives the cookie wall instead of the page.
- Structure: no/multiple h1, heading jumps, no `<main>`, boilerplate ratio, thin content, missing/duplicate titles.
- `RND.CONTENT_BEHIND_INTERACTION` is **always** `not_assessed`, because the collector never clicks. A safety choice, stated honestly.
- **Two eligibility gates on purpose:** shell checks are permissive (a real CSR shell is the page most likely to be misclassified); structure checks are strict (so a cart page isn't flagged for a missing h1).

5.4 structured-data-entity-audit (15 checks)

- **Presence and parsing:** `NO_STRUCTURED_DATA` (checks all three formats), `INVALID_JSONLD` (one syntax error silently voids the whole block).
- **Coverage gated by page type:** it never demands `Product` markup on a blog post. That is the most common false positive in this family.
- `SDE.SCHEMA_VISIBLE_MISMATCH`: the JSON-LD price contradicts the visible price. Same currency required; locale-aware decimal parse (`59,00` equals `59.00`).
- **Identity:** Organization node on the homepage, `sameAs`, a stable `@id`, `ORG_IDENTITY_CONFLICT` (same `@id` on two pages with different telephone/address/host), name consistency (word order and legal suffixes tolerated).
- `ENTITY_AMBIGUITY_RISK` needs **two independent signals**, is marked `inferred`, and is capped at medium.

- **Consistency model:** A consistent · B different but harmless · C genuine contradiction (the **only** category that produces a finding) · D not enough evidence → `not_assessed`.

5.5 answer-retrievability-audit (9 checks)

- Every check is **structural** (position, word-set overlap, digits, punctuation), so they all run in any language.
- The "core fact" on product pages is the page's own structured price. Checks:
  - `NO_DIRECT_ANSWER_LEAD`: the price is buried, or the product isn't named nearby;
  - `SPECS_NOT_EXTRACTABLE`: values stated in prose with no table;
  - `UNITLESS_VALUES`;
  - `ENTITY_NEVER_NAMED`: the body never restates the product's full name;
  - `KEY_FACT_OUTSIDE_MAIN_CONTENT`.
- Long pages: `NO_HEADING_ANCHORS`, `NO_SUMMARY_BLOCK`. FAQ pages: `NO_QA_SURFACE` (headings aren't questions). Sitewide: `NO_CANONICAL_FACT_SURFACE`.

5.6 freshness-corroboration-audit (7 checks)

- `INTRA_SITE_FACT_CONFLICT`: the same **sku** has two prices, or an English-only "tagline is…" phrase has two wordings.
- `DATE_SIGNAL_CONFLICT`: JSON-LD dates, the `Last-Modified` header and sitemap `lastmod` disagree by more than 2 days. Ambiguous formats like `01/02/2026` are skipped, never guessed.
- `STALE_CLAIM` ("latest" on a page older than the rest of the site), `SUPERSEDED_CONTENT_LIVE` (date evidence shows which version is older), `NO_AVAILABILITY_SIGNAL`, `COPYRIGHT_YEAR_LAG` (gap of more than 1 year, so January doesn't trigger it).
- **No off-site corroboration, ever.** Only the collector touches the network. It is recorded in coverage as not assessed.

5.7 engagement-audit (6 checks)

- **Dead ends:** a product, article or docs page with no in-content link (nav and footer links don't count).
- **Continuation:** `NO_RELATED_CONTENT_PATHWAY` (links exist but only lead to utility pages), `WEAK_ACTION_PATHWAY` (a product page has no buy/contact link; English lexicon).
- **Structure:** `EXCESSIVE_PATHWAY_DEPTH` (relative to *this site's own* median depth), `ORPHAN_LIKE_SAMPLED_PAGE` (worded "within the sampled graph").
- **Link text:** `AMBIGUOUS_ACTION_SURFACE` (two "Learn more" links pointing to different places).
- **Analytics boundary:** never claims bounce rate, CTR or conversion. Every finding is a structural observation, not a behavioural prediction.

5.8 opportunity-and-monitoring (synthesis, not detection)

Covers the "beyond the defects found" part of the rubric with five rule families:

1. **Named cross-detector compounds** (6 detector pairs), which require *shared affected URLs*.
2. **Template leverage:** 2+ checks hitting the same pages points to one template fix.
3. **Monitoring** for defect classes that already happened once.
4. **Deferred capabilities:** behaviour, off-site corroboration and real AI citations.
5. **A measurement plan.**

Hard cap of 8, deterministic selection, never invents numbers.

6 · The brain: severity, correlation and priority

All of this lives in `skills/brand-ai-readiness-audit/scripts/merge_findings.py`. Severity describes the damage. Priority describes what to do first. They are computed separately.

6.1 Severity = f(gate_impact, reach, certainty) — merge_findings.py:104

1. **Base from impact:** `blocks` → critical, `degrades` → high, `weakens` → medium.
2. **Reach:** `page` steps down one level; `template` steps down only from critical; `sitewide` stays.
3. **Anti-overclaim caps:** inferred → max high; nondeterministic → max high; medium confidence → max high; low confidence → max medium; the detector's own `severity_ceiling`.
4. **Small-sample cap** (fewer than 3 pages) → max medium. The **exception** is a sitewide, high-confidence origin-level problem (lines 152–168). If a site blocks every bot, only one page is reachable *because of the defect*, and capping it would hide the worst failure.
5. `severity_override` wins outright. That is how the training-only block becomes `info`.

> **Worked example — your real iiitm.ac.in run,** `RND.CSR_SHELL` (browser fallback, 1 high + 2 medium signals):
> 1 high + medium signals → `degrades` → base **high** · template reach keeps it high · confidence medium caps it at high · **result: high**.

6.2 Deduplication — line 177

When two detectors describe the same condition, the authoritative owner keeps the finding and the other becomes a `related_check`. For example, `FRC.NO_CANONICAL_FACT_SURFACE` is kept over `ANS.NO_CANONICAL_FACT_SURFACE`.

6.3 Cross-detector escalation — line 200

`SDE.SCHEMA_VISIBLE_MISMATCH` plus `FRC.INTRA_SITE_FACT_CONFLICT` makes both **critical**. The brand publishes contradictory machine-readable versions of a fact, and no single detector can see that alone.

6.4 Priority is NOT severity — line 214

```
score = severity_weight × reach_weight × effort_weight × (1 + (7 − funnel_stage) × 0.12)

severity_weight : critical 5 · high 3 · medium 1.6 · low 0.7
reach_weight    : sitewide 1.0 · template 0.7 · page 0.4
effort_weight   : trivial 1.0 · config 0.9 · template-change 0.6 · platform-change 0.35
funnel factor   : discovered ×1.84 · accessible ×1.72 · readable ×1.60 · understood ×1.48
                  retrieved ×1.36 · selected ×1.24 · engagement ×1.12

bucket          : ≥4 critical · ≥2 high · ≥0.8 medium · otherwise low
gate shut       : if an access gate is shut, every other finding × 0.35
                  ("can't pay off until the gate opens")
```

Your own iiitm.ac.in output proves this works:

Finding	Severity	Calculation	Priority
`RND.CSR_SHELL`	**high**	3 × 0.7 × 0.35 (platform change) × 1.60 = 1.18	**medium**
`ACC.SITEMAP_MISSING`	**medium**	1.6 × 1.0 × 0.9 (config) × 1.84 = 2.65	**high**
`SDE.NO_STRUCTURED_DATA`	high	3 × 1.0 × 0.6 (template change) × 1.48 = 2.66	high
`ENG.NO_RELATED_CONTENT_PATHWAY`	medium	1.6 × 0.7 × 0.6 × 1.12 = 0.75	low

> A cheap fix at an early gate outranks an expensive fix at a later gate. That is the "fix order comes from the funnel" point.

6.5 Action bands, validation and determinism

- **Action bands** (line 316): 1 Unblock (discovered/accessible) · 2 Make readable & quotable · 3 Keep them & keep it true.
- **Validation** (`validate_report.py:76`):
  - the schema check, plus semantic invariants: counts must equal the findings, ids must be unique, the ISO date needs a `Z`;
  - **evidence-or-silence:** evidence at least 20 characters, and every non-info finding must carry a URL.
- **Determinism:**
  - findings sorted by (severity, funnel stage, check_id, first URL); ids assigned after sorting;
  - no randomness and no model calls in the scoring path;
  - so the same snapshot always gives the same report.

7 · Precision engineering and testing

The false-positive guards you should be able to name

- training vs retrieval roles;
- capability gating (a missing browser is never the site's defect);
- page-type gating;
- language neutrality (structural checks run everywhere, lexical checks only on English pages);
- the two-signal rule for inferential checks;
- severity caps by certainty, and the small-sample cap;
- one dead sitemap entry is noise (needs 3);
- utility pages excluded from noindex;
- an empty `alt=""` counts as a correct decorative marking;
- the 2-day date tolerance.

Fixtures: 35 single-variable mutations

Each fixture differs from one clean site in exactly one respect, so a firing check can be attributed to one cause. The **negative controls** must produce nothing above `info`: F00 clean, F03 training-only block, F08 non-English, F09 two-page site, F10 Microdata-only, F11 correctly noindexed utility pages.

Test layers

Layer	What it checks
L5	Marketplace and Agent Skills spec conformance
L5b	Validator self-test ("a validator that never fails is not a validator")
L6	Catalogue integrity: the build fails if any rule names a check_id nothing can emit
L0	Every fixture builds, serves, and has an expectation manifest
L1	Detector unit tests
L2	Fixture scoring: precision, recall, false positives on negative controls
L3	Determinism: same fixture twice gives identical findings
L4	Every emitted report validates against the schema

Verified on 2026-10-08: ACC 77 · RND 87 · SDE 100 · ANS 75 · FRC 60 · ENG 64 · OPP 62 = **525 unit tests passing**.

8 · Safety and honesty

- **GET/HEAD only, enforced in code:** `fetch()` raises on any other method (`collect.py:130`).
- **Robots and conduct:** robots.txt obeyed including Crawl-delay; honest user-agent; URLs never guessed; nothing clicked, no logins, no challenge solving.

- **Prompt-injection defence:** page text is untrusted data, never an instruction. If a page says "ignore previous instructions", it is recorded, not obeyed (entrypoint contract rule 3).
- **What the report never does:** no commercial levers (paid placement, sponsorships, agencies); never claims a technique guarantees a citation; offers `llms.txt` only as a low-priority experiment with a caveat.

9 · Query Intelligence (optional extension)

A separate subsystem that **reads** `snapshot.json` and `report.json` and never writes back. It is **not in marketplace.json**, and deleting it changes nothing in the audit. It answers: "could this site's own evidence support *this specific* query?"

- **Deterministic NLP (no LLM):** splits a query into intent, entity, attributes, constraints, audience, use case and qualifiers.
  - Word-boundary matching only, so "ram" inside "programmers" doesn't match.
  - "best" is marked *unverifiable*.
  - "cheap" never invents a price ceiling.
- **Six statuses:** SUPPORTED, PARTIALLY_SUPPORTED, NOT_FOUND, CONTRADICTED, NOT_ASSESSABLE and INVALID_REQUIREMENT ("keyboard with RAM": RAM isn't a property of keyboards on this site).
- **Combination checker:** catches the "Frankenstein" answer, where *student* + *under ₹60k* is true of laptop A and *16GB RAM* is true of laptop B, but no single laptop satisfies all three.
- **Five separate scores:** coverage, answerability, confidence, query quality, opportunity. No single blended percentage.

```
Query: "Best laptop for CSE students under ₹60,000 with 16GB RAM"

intent=recommendation · entity=laptop · audience=students
price_max=60000 INR · attribute ram=16 · qualifier best (unverifiable)

entity    → SUPPORTED          audience → SUPPORTED (Value Laptop 14)
price     → SUPPORTED (₹45k)   ram      → SUPPORTED (Pro Laptop X)
best      → NOT_ASSESSABLE
combination → PARTIALLY_SUPPORTED: best single page covers 2 of 3 facts
```

> At the finale, present it as an extension built on the audit's evidence, not as part of the graded marketplace.

10 · Real results you can cite

Site	Result	What it shows
adobe.com `audit-output/SUMMARY.md`	752 pages discovered, 25 sampled. **11 findings, 0 critical, 0 high:** missing `sameAs`, date-signal conflicts, a uniform sitemap `lastmod`, and similar.	The engine **doesn't cry wolf** on a well-built site.
iiitm.ac.in `audit-output/flipkart/`	**15 findings, 3 high:** CSR shell signature (inferred), no Organization node, no structured data at all. Also no sitemap, missing canonicals, specs only in PDFs.	Real defects on a typical institutional site.

11 · Weak spots a judge could find, and honest answers

Better to hear these now than from a judge.

1 · The manifest and entrypoint describe an old scope

`marketplace.json` says freshness does "optional bounded off-site corroboration", describes engagement's old "deep-link landing / interruption" scope, and says opportunities are "archetype-gated". The entrypoint SKILL.md still lists an `external_corroboration` input. The skills' own `checks.md` files say the opposite.

> **Answer:** "That is early planning text. The enforced rule is that only the collector touches the network. Each `checks.md` is generated against the implementation and is authoritative, and off-site corroboration shows up in `not_assessed` on every run."

2 · Escalation is co-occurrence, not same-fact

`escalate()` fires if both check_ids appear *anywhere* on the site, while the severity model says "on the same fact". A *tagline* conflict plus a *price* mismatch would escalate both to critical.

> **Answer:** admit it, and say the next step is keying escalation on a shared sku or URL.

3 · "An invalid report cannot be emitted"

`audit.py` writes `report.json` *before* validating, and on failure only prints `schema valid: NO`.

> **Say instead:** "Validation gates acceptance, the agent path must fix and re-validate, and the runner flags it loudly." Avoid the word "physically", which your video script used.

4 · The "Unblock" band's wording

The band is chosen by funnel stage, so `ACC.CANONICAL_MISSING` sits under "Unblock — these shut a gate".

> **Answer:** "The band shows where in the funnel the problem is. The priority on each action shows how urgent it is."

5 · Robots verdicts are resolved only for path `/`

`verdicts_for(..., "/")` means a rule like `Disallow: /products/` aimed only at OAI-SearchBot is **missed**.

> **Answer:** a known gap; per-template path verdicts are the natural next step.

6 · The CSR-shell fallback can fire on legitimately short pages

A gallery page with analytics scripts trips "thin + script-dominant". Mitigated: confidence is medium, the finding says "inferred", and the fix's first step is "confirm with a browser". In your iiitm run the browser was *installed but failed to launch* ("Executable doesn't exist").

> **Tip:** `playwright install chromium` would give **measured** raw-vs-rendered evidence, which Round 4 named as the ideal evidence. If you use the browser, disclose it.

7 · The localhost / private-IP refusal is an instruction, not code

Phase 0 is written in SKILL.md, but `collect.py` has to accept localhost for the fixtures, so the guard isn't enforced in code.

> **Answer:** be honest; an allowlisted test mode plus a code-level guard is the fix.

8 · Role judgment calls

Amazonbot and Bytespider are labelled training, and Amazonbot also feeds Alexa answers.

> **Answer:** "The role table is data that ages, and unknown tokens produce silence."

9 · The test suite on native Windows

Each request to `localhost` costs about 2 s (measured 2.06 s vs 0.05 s for `127.0.0.1`), so the F03 fixture run timed out. The L2 summary *still printed* "recall 1.00, precision 1.00, 0 false positives" even though nothing was scored.

> **Don't show that line as proof.** Use WSL/Linux/macOS, or cite the 525 unit tests.

10 · Housekeeping

- the README describes a `frontend/` dashboard that isn't in this folder;
- `audit-output/flipkart/` actually holds iiitm.ac.in;
- the local `REPLAY_TEAMNAME.txt` still has `<<FILL>>` placeholders, so make sure your *submitted* copy was filled.

12 · Finale prep

60-second pitch

> "A site can be fully indexed and still never cited, because discovery is a funnel. We built one detector per funnel gate: access, readability, understanding, quotability, freshness, and the on-site next step. All of them read one frozen snapshot, so the audit crawls once, behaves deterministically, and every finding cites a URL and an observed value. Severity is computed from impact × reach × certainty, with caps against overclaiming. Priority is computed separately and ordered by funnel, so the cheap fix at the earliest broken gate comes first. And we engineered against false positives: blocking GPTBot while allowing OAI-SearchBot is reported as a deliberate policy, not a defect, and a negative-control fixture proves it."

Likely questions, with short answers

"Isn't nine skills padding?" No. Each one reads a different evidence slice, owns a different fix and maps to a different funnel gate. The entrypoint owns no checks.

"How did you set severity?" With a formula, not taste: gate impact × reach × certainty, then caps. Show `references/severity-model.md`.

"Why isn't your most severe finding first?" Priority includes effort and funnel position. Use the CSR_SHELL vs SITEMAP_MISSING example from 6.4.

"How do you know it generalizes?" Template-aware sampling, structural rather than lexical checks, page-type gating, honest `not_assessed`, and no site-specific rules.

"Is it deterministic with an LLM in the loop?" The LLM orchestrates. Every verdict comes from deterministic scripts over a frozen snapshot, so the same snapshot gives the same report.

"What if the site blocks you?" The control fetch tells "blocks bots" (a finding) apart from "blocked us" (coverage). The origin-level exception stops the small-sample cap from hiding the block.

"What can't you detect?" Content behind clicks, off-site corroboration, real user behaviour, and path-level robots blocks. Say it plainly.

"How do you handle prompt injection?" Page content is untrusted data. It is stored and measured, never obeyed, and can't change scope, tools, schema or severities.

"What would you build next?" Path-level robots verdicts, escalation keyed on the same fact, a browser by default, multilingual lexicons, and an opt-in off-site corroboration skill.

Demo hazards checklist

- Don't run the fixture suite on native Windows.
- Get the headless browser launching beforehand, or explain the fallback confidently.
- Dry-run your chosen site twice before presenting.
- Don't edit anything under `brand-ai-readiness-audit/`: the engine must match your submission.
- Make sure the model, harness and URL you mention match your REPLAY file exactly.